

exp-mlkem-sepolyvec8-ntt-k4-block

FIPS 203 Alg.13 行 16–17: $k=4$ 八多项式批 NTT

(块紧凑 [HI_8|LO_8] 实现方案)

ascendc 工程 (预研)

2026-06-11

摘要

本文档为 `examples/incubating/exp-mlkem-sepolyvec8-ntt-k4-block/` 的规格说明书(customspec)。算子语义: 对 8 条互不相同、随机生成的 Z_q 系数多项式 ($n=256, q=3329$) 各做 Kyber NTT, 输出 `[8,256] int32`。算法语境对齐 ML-KEM-1024 **K-PKE.KeyGen (Algorithm 13) 行 16–17**: $k=4$ 时 $\hat{s} \leftarrow \text{NTT}(s), \hat{e} \leftarrow \text{NTT}(e)$ 共 8 路批处理。工程壳以已调通探针 `ascendc-tests/fix-merged-kyber-ntt256-limb6-poly8-block-s123/` 为唯一实现参照(块紧凑 Stage1/Stage3 写址; Stage2 与交错版相同)。与 `exp-mlkem-sepolyvec8-ntt-k4` (交错 `[hi0,lo0,...]` 布局) 仅 **GM 行序不同**; 用户实测 SIM 性能无显著差异, 本方案选用 Alg.13 编码一致的块布局。

目录

1 定位与边界	2
1.1 用例性质	2
1.2 与姊妹用例、探针的关系	2
1.3 目标平台	2
2 FIPS 203 Algorithm 13 行 16–17 语义	2
2.1 原文职责 (K-PKE.KeyGen, k 为模块秩)	2
2.2 与 AscendC 张量映射	2
2.3 Stage1 编码与 Alg.13 左矩阵布局	3
3 计算语义: merged_kyber 6bit 三段式 ($k_{\text{polys}}=8$)	3
3.1 GM 行序对照 (实现时禁止混用)	3
3.2 workspace 与 LUT	3
3.3 MIX 同步 (FSM)	4
4 I/O 与 Golden 契约	4
4.1 文件 (<code>gen_data</code> 与 <code>main</code> 逐字一致)	4
4.2 <code>gen_data.py</code> 要求	4
4.3 验收	4

目录	2
5 实现规划	4
5.1 参照文件（照抄顺序）	4
5.2 计划目录	5
5.3 分步验收	5
6 性能与布局选型说明	5
7 已废弃路径（勿 fork）	6
8 关联文档	6

1 定位与边界

1.1 用例性质

- 研究/预研：代码落在 `examples/incubating/exp-*`；验收 **仅 I/O 对拍** (`max_abs_diff=0`)。
- 算法片段：Alg.13 行 16–17 的批 NTT 核；不含行 18 内积、采样、SHA3、完整 KeyGen。
- 输入默认：8 条互异随机 poly（固定 seed）；可选模式将行 0..3 / 4..7 标注为 **s / e**（见 §2）。
- 不在本用例：Encaps/Decaps、FIPS 204、多 NPU 性能调优、Matmul<> 融合路线（已冻结）。

1.2 与姊妹用例、探针的关系

目录	Stage1/3 GM 行序	输入 / 状态
poly8-block-s123 探针	块紧凑 [HI_8 LO_8]	8× 同 poly；CPU+SIM ✓
poly8-s123 探针	交错 [hi0,lo0,...]	8× 同 poly；CPU+SIM ✓
exp-mlkem-sepolyvec8-ntt 交错（照抄 poly8-s123）	交错（照抄 poly8-s123）	8× 异随机；CPU+SIM ✓
本 exp（待实现）	块紧凑（照抄 block-s123）	8× 异随机；方案阶段

1.3 目标平台

- CANN 9.0.0；KernelLaunch (`run.sh cpu/sim/npv`)。
- Atlas A2 (Ascend910B4)；`KERNEL_TYPE_MIX_AIC_1_2`；`blockDim=1`。
- 验证：WSL CPU + `SIM_DIRECT=1 sim`；有实机时补 npv。

2 FIPS 203 Algorithm 13 行 16–17 语义

2.1 原文职责 (K-PKE.KeyGen, k 为模块秩)

- 行 16：对秘密向量 $s = (s_0, \dots, s_{k-1})$ 的每个多项式做 NTT，得 $\hat{s}$ 。
- 行 17：对误差向量 $e = (e_0, \dots, e_{k-1})$ 的每个多项式做 NTT，得 $\hat{e}$ 。
- 本仓首版目标参数： $k=4 \Rightarrow$ 共 8 路 NTT（与 `se_polyvec` 批维一致）。

2.2 与 AscendC 张量映射

- 逻辑输入 `se_polyvec_gm`：形状 `[8,256] int32`，行 i 为 Z_q 系数向量。
- 默认 `gen_data`：8 行两两不等的随机系数（seed 文档化，建议 20260611），不强制与 CBD 采样分布一致——仅验证 NTT 算子 I/O。
- 可选回归 `--alg13-keygen-layout`：行 0..3 标为 s_i ，4..7 标为 e_i ；系数仍由同一 seed 生成，便于与 `ntt_study` 交付件对照命名。
- 输出 `dst / golden`：`[8,256] int32`；第 i 行为 $\text{NTT}(a_i) \bmod q$ 。

2.3 Stage1 编码与 Alg.13 左矩阵布局

Alg.13 行 16–17 之前的编码 (及 exp-mlkem-f203-stage1-encode-vec) 规定左矩阵 `mat_a` [16,256] int8 行序为:

$$\text{行}0..7 = HI(s_0), \dots, HI(s_{k-1}), \quad \text{行}8..15 = LO(s_0), \dots, LO(s_{k-1})$$

即 **先全体 HI**, 再**全体 LO** (块紧凑, 非逐 poly 交错)。本 MIX 核 Stage1 写 S0 须与此一致, 以便后续与 Stage1 纯向量 encode、KeyGen 流水线同一 **GM 契约**。

3 计算语义: merged_kyber 6bit 三段式 ($k_{\text{polys}}=8$)

对每条输入多项式 a_i , 目标 $y_i = \text{NTT}(a_i) \bmod q$ 。拓扑与 merged-kyber-ntt256-limb6 一致, 批维 8:

1. **Stage1 Split (AIV)**: `src` [8,256] int32 $\rightarrow$ S0 [16,256] int8。

- 6bit limb: $hi = \lfloor a/64 \rfloor$, $lo = a - 64 \cdot hi$ ($a \in [0, q)$)。
- 块紧凑写址 (本 exp 与 block-s123 一致): poly p 的 HI 写行 p , LO 写行 $8+p$:

$$S0[p, \cdot] = hi_p, \quad S0[8+p, \cdot] = lo_p.$$

2. **Stage2 Mmad (AIC)**: $A_0 = S0 \cdot M_0$, $A_1 = S0 \cdot M_1$; AicMmad(16,256,256) 两次; 与交错版完全相同 (LUT M4.bin / mat_b_lut_gm 不变)。

3. **Stage3 Merge + Barrett (AIV)**: 从 A0/A1 按块 **Gather** 取行 p 与 $8+p$ (非交错版的 $2p / 2p+1$), 再 Merge + Barrett, 写 `dst` [8,256] int32。

3.1 GM 行序对照 (实现时禁止混用)

张量	块紧凑 (本 exp)	交错 (exp-...-k4)
S0 HI 行	0, 1, ..., 7	0, 2, 4, ..., 14
S0 LO 行	8, 9, ..., 15	1, 3, 5, ..., 15
Merge 读 A0/A1	行 p , $8+p$	行 $2p$, $2p+1$
Stage2 Cube	相同: $M=16, K=N=256$, int8 $\rightarrow$ int32	

数学 NTT 结果一致: 对同一 `src` [8,256], 两种布局仅中间 S0/A0/A1 寻址不同; `dst` 逐行须与 Python 逐行 `ntt_forward` 一致。探针 block-s123 已证: $8\times$ 同 poly 时 $\text{dst} \equiv \text{poly8-s123}$ 。

3.2 workspace 与 LUT

与 block-s123/tiling.h 一致:

$$\text{ws} : M_0, M_1, M_2, M_3 \ (256^2 \text{ int8}), S_0[16, 256], A_0, A_1[16, 256] \text{ int32}.$$

对外 LUT 文件: `input/mat_b_lut_gm.bin`, 形状 [256,512] int8 (四块 256×256 拼接), 与 limb6 / M4.bin 布局兼容。

3.3 MIX 同步 (FSM)

- 单次 launch; CrossCoreSetFlag/CrossCoreWaitFlag 与 merged_kyber / block-s123 相同 (如 kSyncStage1Done=7、kSyncStage2Done=9)。
- 每 AivSplit/AivMerge: 一个 TPipe; tileLength= $8 \times 128 = 1024$; **禁止** for (p){AivSplit...}。

4 I/O 与 Golden 契约

4.1 文件 (gen_data 与 main 逐字一致)

路径	形状	dtype	语义
input/se_polyvec_gm.bin	[8, 256]	int32	8 条互异随机 poly
input/mat_b_lut_gm.bin	[256, 512]	int8	$M_0 \ M_1 \ M_2 \ M_3$
input/tiling.bin	(256, 8)	int32	n 、 k_{polys}
output/output.bin	[8, 256]	int32	内核结果
output/golden.bin	[8, 256]	int32	Python 逐行 NTT

4.2 gen_data.py 要求

1. rng = np.random.default_rng(seed); seed 默认 20260611 (写入 STATUS.md)。
2. 生成 [8, 256], 8 行系数向量两两不等 (assert len(unique)==8)。
3. Golden: 对每行 ntt_sim_kyber.ntt_forward + ntt_test01 交叉断言。
4. 写 LUT: 与 merged-kyber-ntt256-limb6 四块 int8 一致 (同 exp-...-k4)。
5. **禁止** np.tile 同一 poly (那是探针 block-s123 行为, 非本 exp)。

4.3 验收

```
bash run.sh -r cpu -v Ascend910B4
```

```
SIM_DIRECT=1 bash run.sh -r sim -v Ascend910B4
```

```
python3 scripts/verify_result.py output/output.bin output/golden.bin
```

要求 max_abs_diff=0。SIM 须 source scripts/camodel_sim_log.sh, dump 仅在 OP_PROF_*/dump/。

5 实现规划

5.1 参照文件 (照抄顺序)

1. 骨架: 自 ascendc-tests/fix-merged-kyber-ntt256-limb6-poly8-block-s123/ 复制 mmad_custom.cpp、main.cpp、tiling.h、aiv_func.hpp (块版)、ntt_vec.hpp、kyber_limb6.hpp、cmake/、run.sh。

2. 勿用 poly8-s123/aiv_func.hpp(交错写址);勿用 thirdparty/merged_kyber/aiv_func.hpp(单 poly API)。
3. 改 scripts/gen_data.py: 自 exp-mlkem-sepolyvec8-ntt-k4/scripts/gen_data.py 迁入 8 互异随机逻辑; 输入文件名与上表一致。
4. 内核文件名建议:sepolyvec8_ntt_block_custom.cpp(与姊妹 sepolyvec8_ntt_custom.cpp 区分)。

5.2 计划目录

```
exp-mlkem-sepolyvec8-ntt-k4-block/
  exp-mlkem-sepolyvec8-ntt-k4-block-实现方案-customspec.tex / .pdf
  STATUS.md
  sepolyvec8_ntt_block_custom.cpp      # 待: block FSM
  sepolyvec8_ntt_block_custom_tiling.cpp
  main.cpp, CMakeLists.txt, run.sh
  aiv_func.hpp, ntt_vec.hpp, kyber_limb6.hpp, tiling.h
  scripts/gen_data.py, verify_result.py
```

5.3 分步验收

步	内容	验收
S0	本 customspec PDF	文档就绪
S1	gen_data: 8 互异随机 + golden	逐行 NTT 一致
S2	迁入 block-s123 内核 + 改输入路径	CPU max_diff=0
S3	SIM 单趟 MIX launch	无挂死; dump 路径合规
S4	与 exp-...-k4 同 src 对拍	两 exp dst 逐行相同
S5	A2 npu	后续

6 性能与布局选型说明

- 用户对比 poly8-block-s123 与 poly8-s123 的 SIM: 模拟性能无显著差异。
- 选用块紧凑布局的理由: 与 Alg.13 行 16–17 前编码(mat_a [HI(8),LO(8)]),exp-mlkem-f203-stage1-enc ntt_study F203 交付语义一致; 避免 KeyGen 流水线中 Stage1 encode 与 Stage2/NTT 混用两种行序。
- exp-mlkem-sepolyvec8-ntt-k4 (交错版) 保留为对照与回归; 本 exp 为 KeyGen 路径的推荐批 NTT 实现。

7 已废弃路径 (勿 fork)

- NTT 内 Matmul<> / 两段式 Stage2: 见 `ascendc-tests/frozen/.qa/2026-06/2026-06-11-NTT-Matmul` 路线废弃冻结.md。
- [16,512] RouteA 宽表作 Stage2 输出 (ONNX 语义) 与本 limb6 紧凑 [16,256] 契约不同。

8 关联文档

- 探针: `ascendc-tests/fix-merged-kyber-ntt256-limb6-poly8-block-s123/STATUS.md`
- 姊妹 exp: `examples/incubating/exp-mlkem-sepolyvec8-ntt-k4/`
- Stage1 encode: `exp-mlkem-f203-stage1-encode-vec-实现方案-customspec.pdf`
- 策略: `docs/research/20260610-f203-ntt-mix-merged-kyber 开发策略.md`